



NAME: OKIROR SIMON PETER

REG NO. 2025/BCS/160/PS

COURSE: DATABASE SYSTEMS

LECTURER: KENNETH BAGUMA

1.EXECUTIVE SUMMARY

This project presents a Tourism Reservation Database System designed to manage customer bookings for tours, hotels, transport, and payments. The system was developed using MySQL and aims to improve efficiency in handling tourism operations by centralizing data management.

The system enables customers to register, book tours, select hotels, arrange transport, and make payments in an organized and efficient manner. It demonstrates key database concepts such as entity relationships, normalization, and SQL query implementation.

2. BUSINESS CASE

2.1 Application Domain

The system is designed for the tourism and travel industry. It supports operations such as customer registration, tour booking, hotel reservation, transport arrangement, and payment processing.

The system serves

- Customers (booking services)
- Administrators (managing bookings and data)

2.2 Benefits

Tangible Benefits

- Reduced paperwork
- Faster booking process
- Efficient data storage

Intangible Benefits

- Improved customer satisfaction
- Better decision-making
- Increased service quality

2.3 Costs

Tangible Benefits

- Software installation (MySQL/XAMPP)
- Computer hardware

Intangible Benefits

- User training
- Development time

User issues

- Users may require training to effectively use the system. Some users may lack technical skills, which may affect system adoption at early stages.

2.4 User Risks

Users may require training to effectively use the system. Some users may lack technical skills, which may affect system adoption at early stages.

2.5 risks

- ❖ Data loss or corruption
- ❖ System failure
- ❖ Security threats

2.6 Limitations

- ❖ Requires a computer system
- ❖ Limited offline access
- ❖ Suitable mainly for small-scale implementation

3. Design

3.1 Logical Design

Entities

-  Customer
-  Tour
-  Hotel
-  Booking
-  Payment
-  Transport

Entities and Attributes

Customer

- CustomerID (primary Key)
- Name
- Email
- Phone
- Address

Tour

- TourID (primary Key)
- Destination
- Price

- Duration

Hotel

- HotelID (primary Key)
- Name
- Location
- PricePerNight

Booking

- BookingID (Primary Key)
- CustomerID (Foreign Key)
- TourID (Foreign Key)
- HotelID (Foreign Key)
- BookingDate

Payment

- PaymentID (Primary Key)
- BookingID (Foreign Key)
- Amount
- PaymentDate
- Status

Transport

- TransportID (Primary Key)
- Type
- Cost
- BookingID (Foreign Key)

Each entity in the system represents a real-world object within the tourism domain.

Attributes describe the properties of each entity, while primary keys uniquely identify each record.

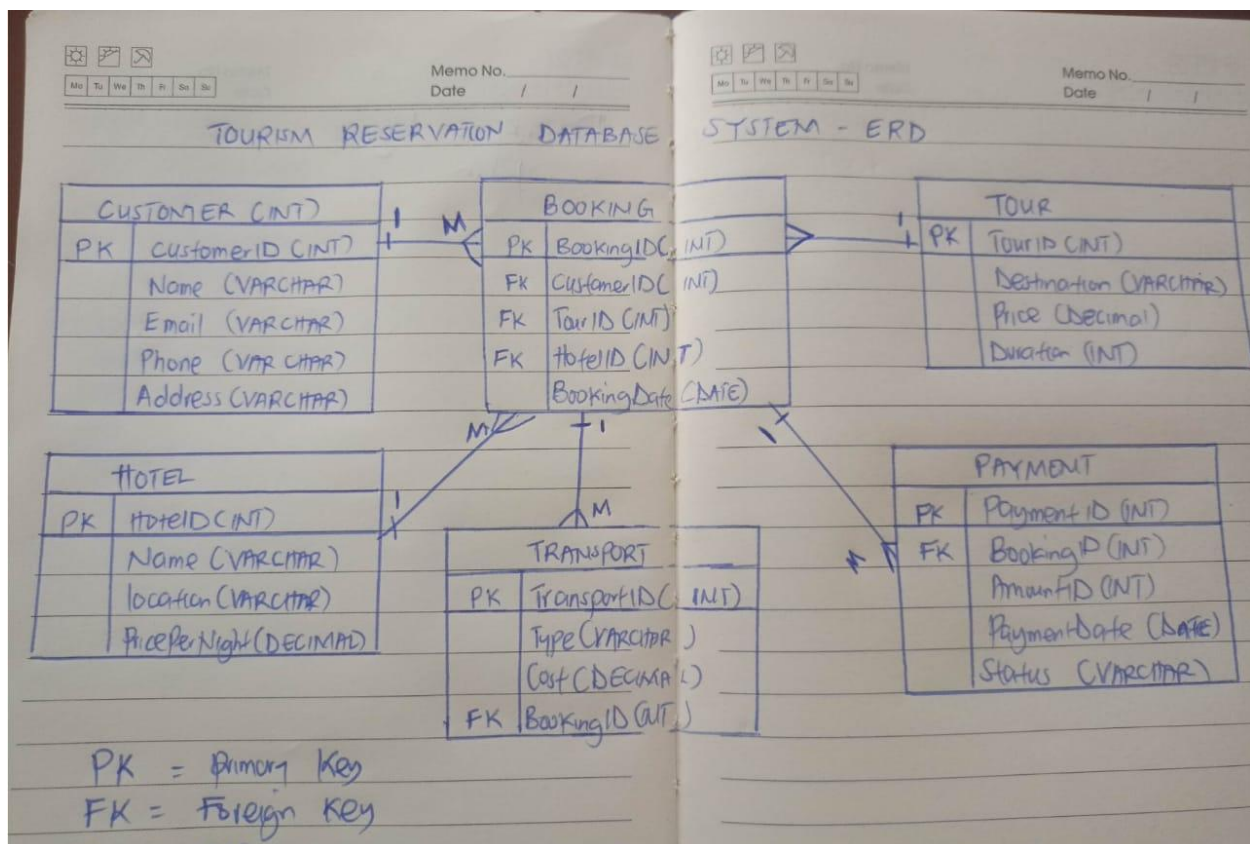
Foreign keys are used to establish relationships between entities, ensuring consistency and integrity of data.

Relationships and Multiplicity

- One Customer → Many Booking (1:M)
- One Tour → Many Booking (1:M)
- One Hotel → Many Booking (1:M)
- One Booking → Many Payment (1:M)
- One Booking → Many Transport (1:M)

ERD

The system uses the Booking entity as the central link connecting Customer, Tour, and Hotel. It also links to Payment and Transport entities. This design ensures efficient data organization and reduces redundancy.



3.2 User Interface Design

The system interface consists of

- Login Page
- Customer Registration Page
- Booking Page
- Tour Selection Page
- Payment Page
- Admin Dashboard

These interfaces allow users to interact with the system easily.

3.3 Physical Design

The database schema consists of six tables

Customer, Tour, Hotel, Booking, Payment, and Transport.

Relationships are implemented using foreign keys to maintain data integrity.

3.4 Database Normalization

The database was designed using normalization principles to reduce redundancy and improve data integrity.

Normalization ensures that data is stored in separate tables based on logical groupings and relationships.

For example, customer information is stored in the Customer table instead of being repeated in the Booking table.

This reduces duplication and makes updates easier and more consistent.

The system follows basic normalization rules, ensuring that each table contains data related to a single entity.

3.5 Database schema

The database schema defines the structure of the database, including tables, attributes, and relationships.

Each table is uniquely identified by a primary key, while foreign keys are used to link related tables.

For example, the Booking table contains foreign keys referencing Customer, Tour, and Hotel tables.

This ensures referential integrity, meaning that related data must exist before a record can be created.

3.6 System Requirements

The system was developed based on specific functional requirements.

It must allow users to register, create bookings, select tours and hotels, arrange transport, and make payments.

Non-functional requirements include data security, system reliability, and ease of use.

These requirements guided the design and implementation of the database system.

3.7 Queries and Reports

The system supports queries such as

- Viewing customer details
- Viewing bookings
- Viewing payment information
- Generating full booking reports using JOIN queries

These outputs help users and administrators make informed decisions.

4 IMPLEMENTATION

The system was implemented using MySQL in XAMPP.

4.1 Table Creation

Tables were created using SQL commands based on the logical design.

4.2 Data Insertion

Sample data was inserted into all tables to test the system

4.3 System Evaluation

The system performed successfully during testing.

All tables were properly linked, and queries returned accurate and expected results.

The JOIN query demonstrated the ability to combine multiple tables into a single output.

This confirms that the system meets its objectives and can effectively manage tourism booking operations.

4.4 Challenges faced

During the development process, several challenges were encountered.

These included errors during table creation, especially when defining foreign key relationships.

For example, one table initially failed to function correctly and had to be dropped and recreated.

These challenges were resolved through debugging and repeated testing.

4.5 Future Improvements

The system can be improved in several ways in the future.

A web-based or mobile interface can be developed to allow users to access the system online.

Security features such as user authentication and data encryption can also be added.

Additionally, the system can be expanded to support larger organizations with more complex requirements.

4.7 Sample Queries and Results

SHOW TABLES


```
XAMPP for Windows - mysql
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> USE tourism_db;
Database changed
MariaDB [tourism_db]> SHOW TABLES;
+-----+
| Tables_in_tourism_db |
+-----+
| booking               |
| customer              |
| hotel                 |
| payment               |
| tour                  |
| transport              |
+-----+
6 rows in set (0.023 sec)
```

This screenshot shows the result of the `SHOW TABLES` command executed in the database. It displays all the tables created in the `tourism_db` database, including Customer, Tour, Hotel, Booking, Payment, and Transport. This confirms that the database structure has been successfully implemented according to the system design.

Customer Table Output

```
MariaDB [tourism_db]> SELECT * FROM Customer;
```

CustomerID	Name	Email	Phone	Address
1	okiror	okirors@gmail.com	0700000000	kamplala
2	Simon Peter	simon@email.com	0700000000	Mbarara

This screenshot shows the contents of the Customer table.

It stores important customer information such as CustomerID, name, email, phone number, and address.

For example, each row represents a different customer who can make bookings in the system.

This table ensures that customer data is properly stored and can be easily retrieved when needed.

Tour table output

```
MariaDB [tourism_db]> SELECT * FROM Tour;
+-----+-----+-----+-----+
| TourID | Destination          | Price  | Duration |
+-----+-----+-----+-----+
|      1 | Queen Elizabeth Park | 500.00 |      3   |
+-----+-----+-----+-----+
1 row in set (0.000 sec)
```

This screenshot shows the Tour table, which contains information about available tours.

It includes details such as destination, price, and duration.

For example, a tour to Queen Elizabeth National Park is stored with its cost and number of days.

This table allows the system to manage and display different tourism options.

Hotel table output

```
MariaDB [tourism_db]> SELECT * FROM Hotel;
+-----+-----+-----+-----+
| HotelID | Name          | Location | PricePerNight |
+-----+-----+-----+-----+
|      1 | Lake View Hotel | Mbarara  |      100.00   |
+-----+-----+-----+-----+
1 row in set (0.000 sec)
```

This screenshot shows the Hotel table.

It stores information about hotels available for booking, including hotel name, location, and price per night.

For example, a hotel in Mbarara is listed with its pricing details.

This allows customers to choose accommodation when making bookings.

Booking Table Output

```
MariaDB [tourism_db]> SELECT * FROM Customer;
```

CustomerID	Name	Email	Phone	Address
1	okiror	okirors@gmail.com	0700000000	kampla
2	Simon Peter	simon@email.com	0700000000	Mbarar

This screenshot shows the Booking table, which is the central table in the system.

It links customers, tours, and hotels using foreign keys.

For example, it shows which customer made a booking, the selected tour, the chosen hotel, and the booking date.

This table is essential because it connects all major components of the system.

Transport Table Output

```
MariaDB [tourism_db]> SELECT * FROM Transport;
```

TransportID	Type	Cost	BookingID
1	Bus	50.00	1

1 row in set (0.012 sec)

This screenshot shows the Transport table.

It stores details about transport arrangements associated with bookings.

For example, it shows the type of transport used, such as a bus, and the cost.

This ensures that transport services are properly recorded and linked to bookings.

Payment table output

```
MariaDB [tourism_db]> SELECT * FROM Payment;
```

PaymentID	BookingID	Amount	PaymentDate	Status
1	1	500.00	2026-05-05	Paid

```
1 row in set (0.000 sec)
```

This screenshot shows the Transport table.

It stores details about transport arrangements associated with bookings.

For example, it shows the type of transport used, such as a bus, and the cost.

This ensures that transport services are properly recorded and linked to bookings.

JOIN Q Query Results

```

MariaDB [tourism_db]> SELECT
->     Customer.Name,
->     Tour.Destination,
->     Hotel.Name AS Hotel,
->     Booking.BookingDate,
->     Payment.Amount,
->     Transport.Type
-> FROM Booking
-> JOIN Customer ON Booking.CustomerID = Customer.CustomerID
-> JOIN Tour ON Booking.TourID = Tour.TourID
-> JOIN Hotel ON Booking.HotelID = Hotel.HotelID
-> JOIN Payment ON Payment.BookingID = Booking.BookingID
-> JOIN Transport ON Transport.BookingID = Booking.BookingID;
+-----+-----+-----+-----+-----+-----+
| Name | Destination | Hotel | BookingDate | Amount | Type |
+-----+-----+-----+-----+-----+-----+
| okiror | Queen Elizabeth Park | Lake View Hotel | 2026-05-05 | 500.00 | Bus |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.026 sec)

```

The JOIN query combines data from multiple tables to produce a complete view of the booking process.

It retrieves customer information, tour details, hotel data, payment records, and transport details in a single result.

This demonstrates how relational databases allow data to be linked and retrieved efficiently.

The successful execution of this query confirms that all relationships between tables are correctly implemented.

Bibliography

Elmasri & Navathe – Database Systems

MySQL Official Documentation

Lecture Notes

Online tutorials on relational databases